

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»

Кафедра цифрових технологій в енергетиці

Розрахункова-графічна робота

На тему: “Просторове аудіо”

З дисципліни “Методи синтезу віртуальної реальності”

Варіант 20

Виконав:

Студент гр. ТР-51мп, НН ІАТЕ

Лудченко А.Є.

Перевірив:

Демчишин А. А.

Київ – 2026

Розділ 1. Опис завдання

1.1 Огляд

Метою цього завдання є розширення існуючого WebGL-додатку для підтримки просторового аудіо з використанням Web Audio API. Основна задача полягає в реалізації обертання джерела звуку навколо геометричного центру нерухомої параметричної поверхні за допомогою відчутного (tangible) інтерфейсу, в ролі якого виступають сенсори смартфона.

Функціональні вимоги до проєкту передбачають відтворення музичного файлу у форматі mp3, просторова позиція якого контролюється користувачем у реальному часі. Необхідно візуалізувати цю позицію за допомогою 3D-сфери на сцені. Додатково вимагається імплементація звукового фільтра, а саме режекторного (Notch) фільтра, з елементами інтерфейсу для його увімкнення, вимкнення та динамічного налаштування параметрів частоти.

Кінцевим результатом є інтерактивна 3D-візуалізація зі стереоскопічним ефектом, де користувач може фізично маніпулювати положенням джерела звуку в просторі за допомогою мобільного пристрою та чути відповідні просторові та частотні зміни у відтворюваній аудіокомпозиції.

Розділ 2. Теоретичні відомості

2.1 Основи просторового аудіо

Просторове аудіо дозволяє імітувати розташування джерела звуку в тривимірному просторі відносно слухача. У вебсередовищі це реалізується за допомогою Web Audio API, зокрема вузла PannerNode. Цей вузол використовує алгоритми HRTF (Head-Related Transfer Function) для створення бінаурального ефекту, обчислюючи, як звук досягає лівого та правого вуха з урахуванням акустичної тіні голови, форми вушних раковин та затримок у часі. Для досягнення максимальної реалістичності також враховується модель загасання звуку з відстанню, найчастіше обернено пропорційна, що імітує природне поширення звукових хвиль у відкритому середовищі.

2.2 Режекторний фільтр

Режекторний фільтр, також відомий як Notch-фільтр, є різновидом смугового фільтра, який різко послаблює амплітуду сигналів у дуже вузькій смузі частот, залишаючи всі інші частоти без значних змін. У Web Audio API для його реалізації використовується інтерфейс BiquadFilterNode із типом notch. Важливим параметром цього фільтра є добротність, яка визначає ширину смуги пригнічення: чим вища добротність, тим вужча і глибша "яма" на амплітудно-частотній характеристиці. Змінюючи центральну частоту цього фільтра в режимі реального часу, можна досягти ефекту динамічного поглинання певних інструментів або вокалу в музичному міксі, створюючи ефект "фазера" або відчуття знаходження під водою, що суттєво розширює можливості звукового дизайну.

2.3 Синхронізація координат

Для коректної роботи системи необхідно, щоб координати візуальної

3D-сфери збігалися з координатами аудіоджерела. Це досягається шляхом застосування складних матричних трансформацій до початкового вектора позиції. Спочатку нормалізовані вектори з магнітометра смартфона перетворюються на базову матрицю обертання. Далі ця матриця повороту множиться на матрицю паралельного перенесення, яка визначає радіус орбіти звукової сфери. Отримана таким чином локальна матриця трансформації дозволяє обчислити фінальні 3D-координати за допомогою спеціальних математичних функцій бібліотеки, після чого ці координати передаються безпосередньо у параметри позиціювання вузла `PannerNode`, гарантуючи абсолютну синхронізацію візуального та аудіального рядів.

Розділ 3. Деталі реалізації

3.1 Огляд архітектури

Додаток побудовано на основі нативного WebGL для відображення 3D-графіки та потужного інтерфейсу Web Audio API для високопродуктивної обробки звуку в браузері. Програмна архітектура чітко розділена на кілька логічних модулів, що забезпечують незалежну, але синхронізовану роботу візуального та звукового рушіїв. Дані просторової орієнтації смартфона, необхідні для відчутного інтерфейсу, передаються на клієнт у форматі JSON через високошвидкісне WebSocket-з'єднання, що гарантує мінімальну затримку відгуку. Логіка додатку об'єднує графічний рушій, який відповідає за генерацію геометрії та рендеринг анагліфного стереозображення для двох очей, та аудіо-рушій, що відповідає за динамічну маршрутизацію вузлів звукового графа.

3.2 Налаштування аудіо графа

У файлі головного скрипта ініціалізується базовий екземпляр AudioContext, який є фундаментом для будь-яких операцій зі звуком. Завантаження музичного файлу відбувається асинхронно за допомогою Fetch API, після чого отримані бінарні дані декодуються у формат AudioBuffer, готовий до відтворення. Для позиціонування звуку в просторі створюється вузол PannerNode з налаштуваннями моделі панорамування HRTF та моделлю загасання за обернено пропорційним законом. Для частотної обробки створюється BiquadFilterNode типу notch з базовою добротністю, що дозволяє вирізати вузьку смугу частот. Під час увімкнення або вимкнення фільтра через інтерфейс відбувається динамічна перекомпозиція аудіографа: активні з'єднання розриваються методом disconnect, після чого джерело підключається безпосередньо до паннера, паннер — до фільтра, а фільтр —

до системного виходу аудіоконтексту. Керування станом відтворення реалізовано через виклики методів `suspend` та `resume`, що дозволяє ставити музику на паузу без втрати поточної позиції треку.

3.3 Оновлення частоти фільтра

Для запобігання звуковим артефактам, таким як "клацання" при різкій зміні частоти фільтра, використовується метод `setTargetAtTime`. Він забезпечує експоненційне згладжування переходу між поточним та новим значенням частоти при русі повзунка в інтерфейсі користувача.

3.4 Синхронізація у циклі рендерингу

У головному циклі відмальовування вираховується комбінована матриця обертання, що базується на положенні трекболу та даних з магнітометра. На основі цієї матриці розраховується матриця локальної позиції звукової сфери. За допомогою математичної функції трансформації точки локальні координати вилучаються з матриці та присвоюються властивостям `positionX`, `positionY` та `positionZ` просторового аудіовузла. Окремо рендеряться стаціонарна поверхня `Neovius` та рухома сфера-джерело звуку.

Розділ 4. Інструкція користувача

4.1 Огляд інтерфейсу

Після запуску додатку у вікні браузера відображається 3D-сцена з параметричною поверхнею. Над сценою розміщена панель керування, поділена на кілька блоків. Перший блок дозволяє підключитися до сенсорів смартфона за IP-адресою. Другий блок містить кнопку керування відтворенням, прапорець для активації режекторного фільтра та повзунок динамічної зміни частоти. Третій блок дозволяє налаштовувати параметри стереокамери.

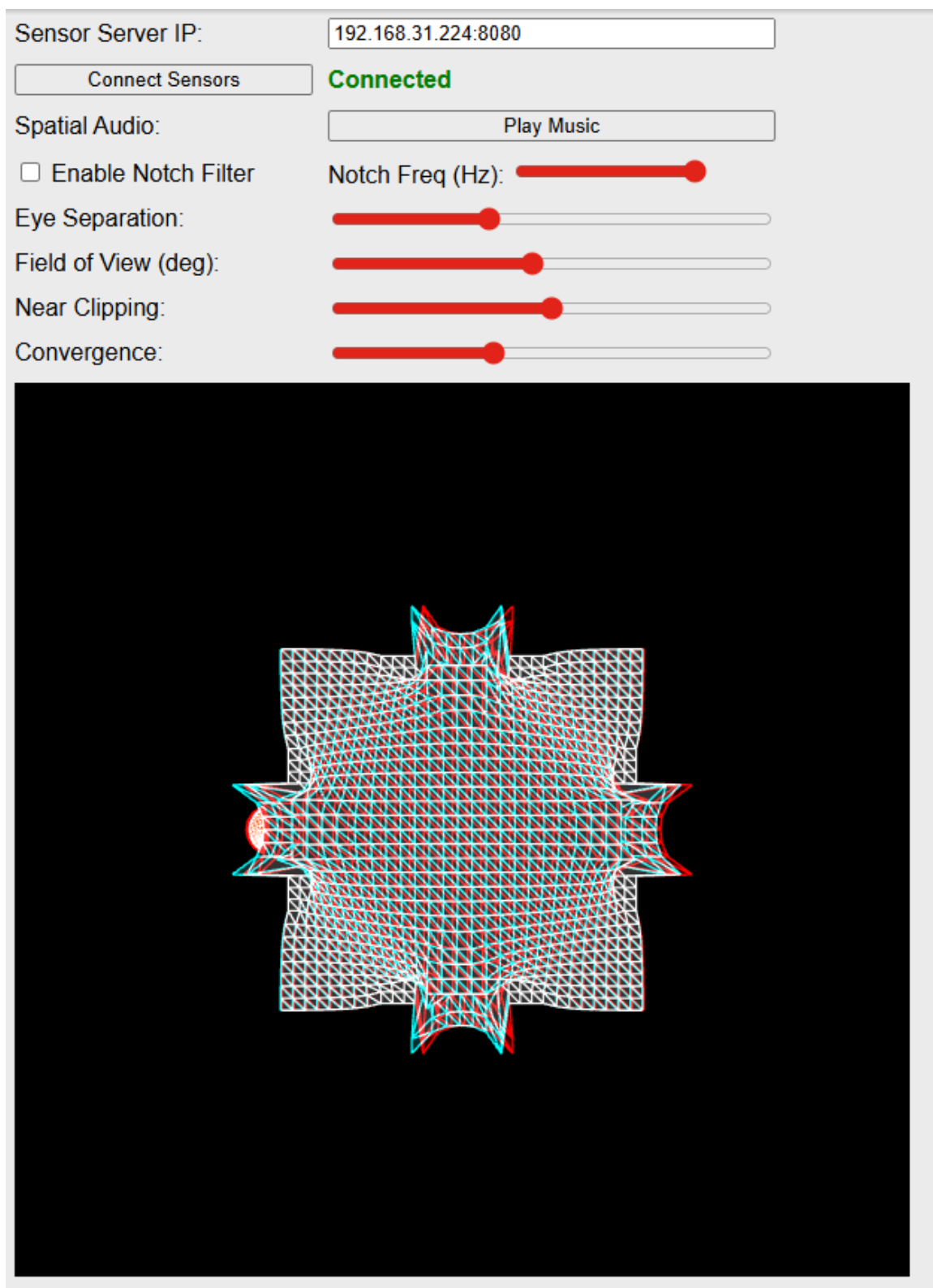


Рисунок 4.1 - Початковий вигляд додатку

4.2 Керування додатком

Для початку роботи з просторовим звуком необхідно натиснути кнопку відтворення музики. Обертання сцени здійснюється за допомогою миші (перетягування по полотну) або через обертання підключеного смартфона в просторі. При цьому помаранчева сфера почне обертатися навколо центру, і звук плавно переміщуватиметься між лівим та правим навушником.

4.3 Робота з фільтром

Для перевірки роботи режекторного фільтра необхідно увімкнути відповідний прапорець. Після цього, плавно переміщуючи повзунок частоти в діапазоні від 200 до 5000 Гц, можна спостерігати, як з музичної композиції вирізаються різні частотні смуги, поглинаючи бас на низьких значеннях або вокал на середніх.

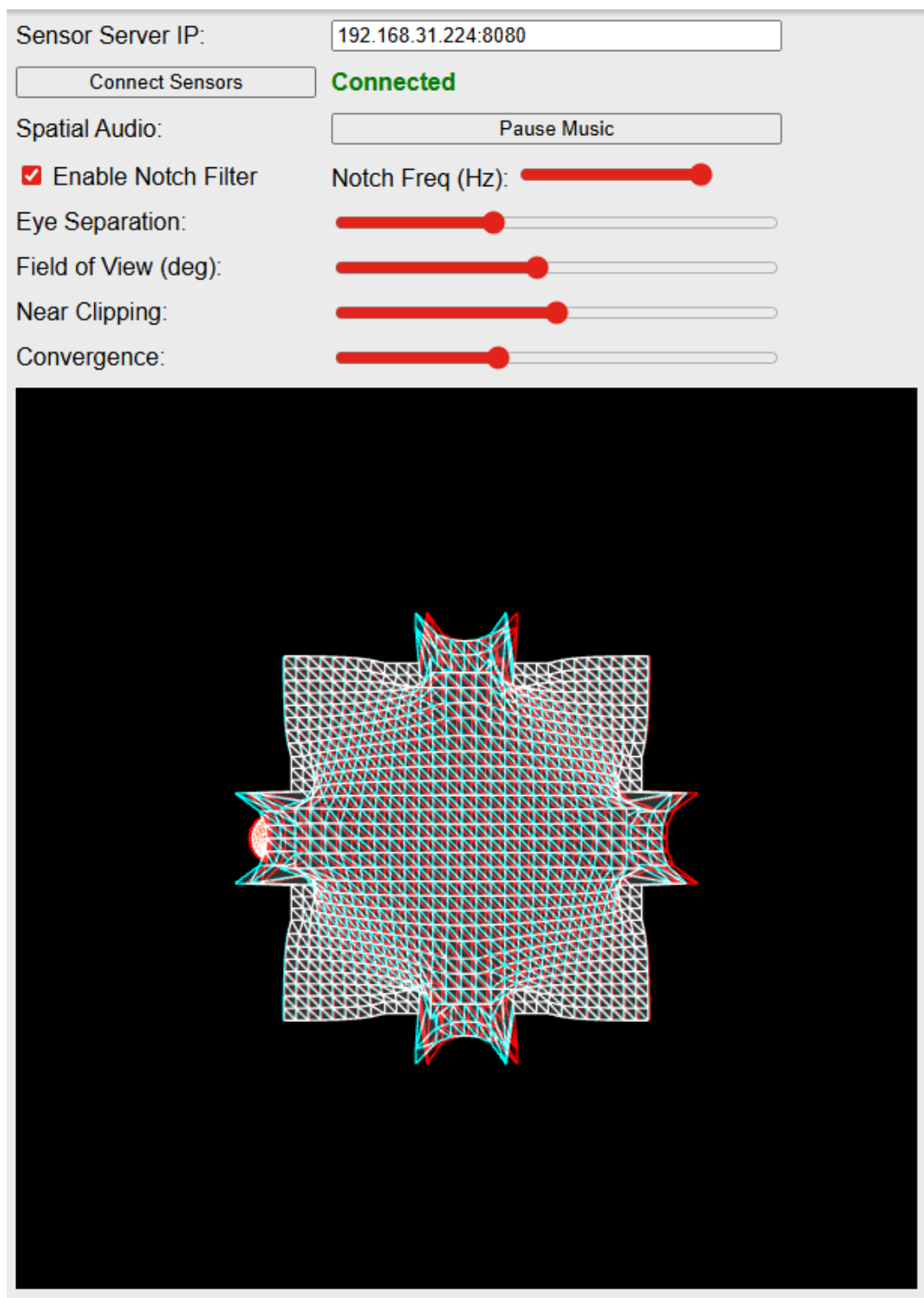


Рисунок 4.2 - Текстура, обернута навколо зміщеної точки

Розділ 5. Зразок вихідного коду

5.1 Ініціалізація аудіо та фільтра (main.js)

Цей розділ відповідає за основну математику трансформації.

```
async function initAudio() {  
  if (audioCtx) return;  
  
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();  
  
  panner = audioCtx.createPanner();  
  panner.panningModel = "HRTF";  
  panner.distanceModel = "inverse";  
  panner.refDistance = 1;  
  panner.maxDistance = 10000;  
  panner.rolloffFactor = 1;  
  
  filter = audioCtx.createBiquadFilter();  
  filter.type = "notch";  
  filter.frequency.value =  
Number(document.getElementById("notchFreq").value);  
  filter.Q.value = 1;  
  
  const response = await fetch("./music.mp3");  
  const arrayBuffer = await response.arrayBuffer();  
  audioBuffer = await audioCtx.decodeAudioData(arrayBuffer);  
}
```

5.2 Динамічне оновлення частоти фільтра (main.js)

Ця функція JavaScript перехоплює введення користувача та оновлює центральні координати.

```
function updateNotchFreq(value) {  
  if (filter && audioCtx) {
```

```
    filter.frequency.setTargetAtTime(Number(value), audioCtx.currentTime,
0.05);
  }
}
```

5.3 Синхронізація позиції звуку з 3D-графікою (main.js)

Передача оновлених даних до GPU.

```
let modelView = spaceball.getViewMatrix();
let combinedRotation = m4.multiply(modelView, sensorRotationMatrix);

let soundLocalRotation = m4.multiply(m4.identity(), combinedRotation);
let orbitRadiusTransform = m4.translation(4, 0, 0);
    let    soundPositionLocal    =    m4.multiply(soundLocalRotation,
orbitRadiusTransform);

let localPos = m4.transformPoint(soundPositionLocal, [0, 0, 0]);
if (panner && audioCtx) {
  panner.positionX.value = localPos[0];
  panner.positionY.value = localPos[1];
  panner.positionZ.value = localPos[2];
}
```